

第10讲

位置编码：Transformer

为什么必须知道“谁在前、谁在后”？

从“狗咬人 $\neq$ 人咬狗”，到 Sinusoidal、相对位置与 RoPE

本讲目标

学完后，你应该能解释：为什么纯 Self-Attention 本身没有天然顺序感；位置编码到底补了什么信息；绝对位置与相对位置有什么区别；Sinusoidal Position Encoding 的直觉；RoPE 为什么通过旋转 Q/K 引入位置信息；以及 Position Encoding 与 Causal Mask 为什么不是一回事。

没有位置信息时：Self-Attention 没有天然的“第几个位置”坐标



如果只依据 token 内容做 Q/K/V，公式里没有一个独立变量告诉模型“这是第 1、2、3 个位置”

先记一句话

Attention 告诉模型“该看谁”；Position Information 告诉模型“谁在哪里、彼此相距多远”。

1. 为什么顺序对语言如此重要？

中文里，“狗咬人”和“人咬狗”包含几乎相同的三个字，但事件含义完全不同。自然语言不是“词袋”，顺序往往决定主谓宾、时间先后、因果关系和指代关系。

句子	词集合	真实含义
狗咬人	{狗，咬，人}	狗是施事者，人是受事者
人咬狗	{人，咬，狗}	人是施事者，狗是受事者

问题

如果 Transformer 只知道每个 token 的内容向量，却没有任何位置线索，它怎样区分“谁在前、谁在后”？

2. 为什么纯 Self-Attention 没有天然的顺序坐标？

回忆上一讲：Self-Attention 主要做 Q/K/V 投影、点积、Softmax 和对 V 加权。这里的公式本身只比较“内容表示”，并没有一个专门的变量说“这是第 3 个 token”。

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

没有位置信息时：Self-Attention 没有天然的“第几个位置”坐标



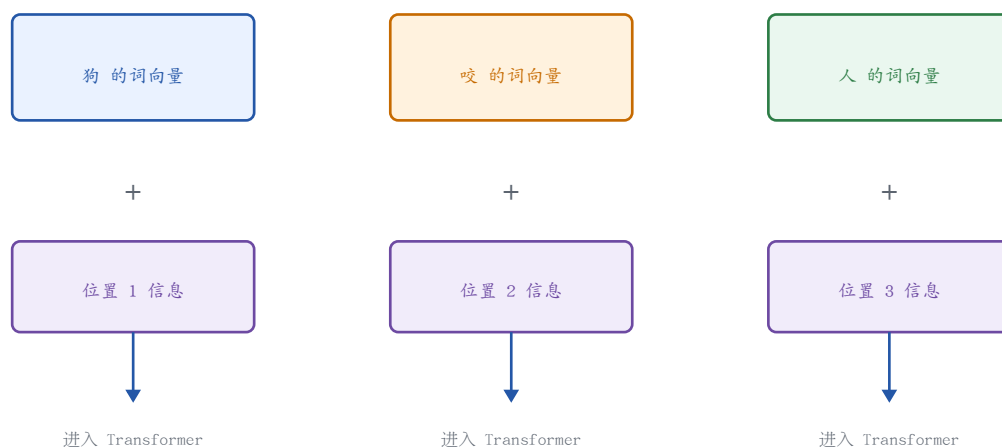
如果只依据 token 内容做 Q/K/V，公式里没有一个独立变量告诉模型“这是第 1、2、3 个位置”

更严谨一点

在没有位置信息时，Self-Attention 对输入 token 的排列具有置换等变性（permutation equivariance）：你把输入位置整体重新排列，输出也会按同样方式重新排列；架构本身没有固定“第几个位置”的坐标系。

3. 最直观的补救：给每个 token 加上“位置身份证”

最直接的办法：Token Embedding + Position Information



最经典的想法非常直接：一个 token 进入 Transformer 前，不只携带“我是谁”的 Token Embedding，还携带“我在哪里”的 Position Information。

```
x_i = token_embedding_i + position_embedding_i
```

通俗类比

Token Embedding 像“姓名与个人资料”；Position Encoding 像“座位号”。同一个人坐在第 1 排和第 20 排，身份没变，但空间位置不同。

4. Learned Positional Embedding：位置也可以当参数来学

一种很直观的做法是：给位置
0、1、2、3……各自准备一个可学习向量。训练开始时随机，训练过程中和其他参数一起由梯度更新。

位置	可学习向量（示意）
0	[0.12, -0.31, ...]
1	[0.44, 0.08, ...]
2	[-0.21, 0.57, ...]
...	...

优点

简单、直接，让模型自己学习“位置 0”和“位置 50”应怎样表示。

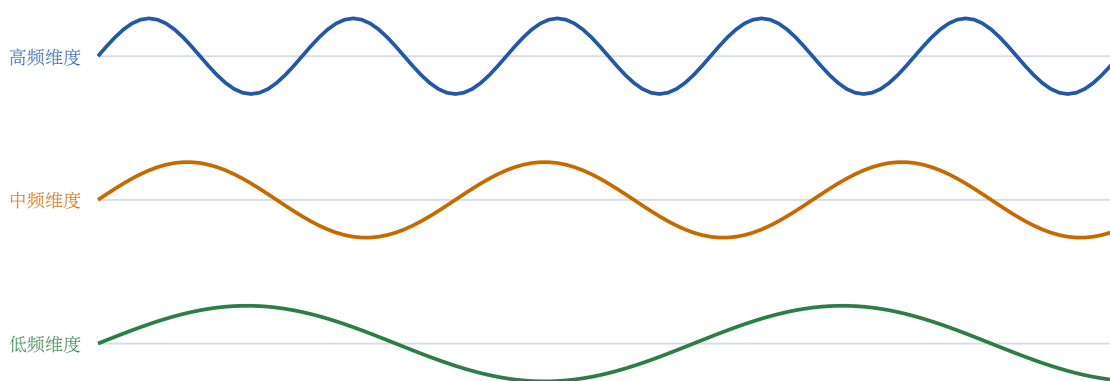
局限直觉

如果训练时只学过有限位置，超过已学习位置范围时如何外推就会变得不自然。

5. Sinusoidal Positional Encoding：为什么用正弦和余弦？

原始 Transformer 论文提出了一种不需要学习参数的位置表示：让不同维度使用不同频率的 $\sin/\cos$ 函数。每个位置会得到一组独特数字。

Sinusoidal Positional Encoding：不同维度使用不同频率的正弦/余弦



多种频率组合后，每个位置形成独特的“位置指纹”

经典形式（只需认得，不要求背）：

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_{model})}) \quad PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_{model})})$$

6. Sinusoidal

的核心直觉：多把“不同刻度的尺子”同时测位置

只用一个周期的正弦波会重复：走得足够远以后数值又相似。解决办法是让很多维度拥有不同频率。高频维度对近距离变化敏感，低频维度能覆盖更长尺度。

- 高频：像毫米刻度，局部位置变化时数值变化快；
- 低频：像公里刻度，变化慢但覆盖范围长；
- 许多频率组合：共同形成位置的“多尺度指纹”。

不要误解

模型不是把 $\sin/\cos$ 当作“数字 37”的人类意义去理解；这些值只是进入网络的数值特征，后续层会学习如何利用它们。

7. 绝对位置 vs 相对位置：两个不同的问题

问题	绝对位置回答	相对位置回答
“你在哪里？”	我是第 128 个 token	–
“你离我多远？”	需要模型间接推断	你在我前面 7 个 token
语言里常见需求	段首/段尾、固定位置模式	局部依赖、距离、顺序关系

为什么相对位置很自然

很多语言关系更关心“相距几步”，而不是“绝对是第 1247 和第 1254 位”。

位置方法的三类直觉



用于建立心智模型；真实实现还存在许多工程变体和组合

8. Relative Position / Bias：把距离直接放进注意力分数

另一类方法不是先给 token 加一个位置向量，而是在 Attention 的分数阶段，根据两个位置的相对距离增加一个偏置项。概念上可写成：

$$\text{score}(i,j) = q_i \cdot k_j + \text{position_bias}(i,j)$$

这样，“当前位置 i 看位置 j ”时，模型不仅知道内容相似度，也可以得到与距离/方向有关的额外信号。

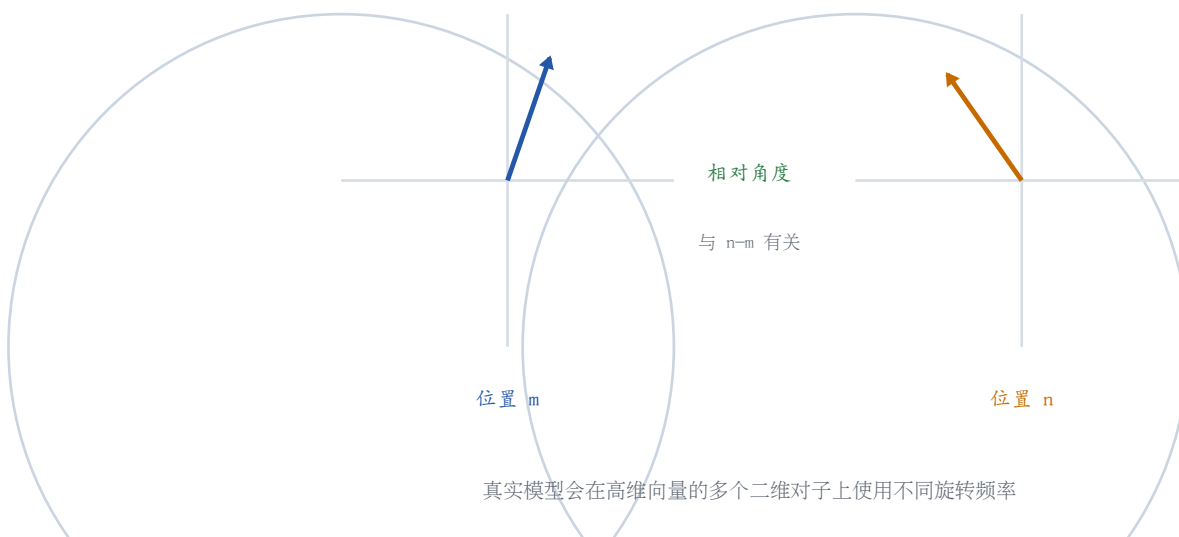
直觉

内容分数像“这个人和我专业是否相关”；位置 bias 像“他离我多远”。最终注意力综合两种信息。

9. RoPE：现代 LLM 里非常重要的位置方法

RoPE (Rotary Position Embedding, 旋转位置编码) 的关键直觉是：按 token 的位置，对 Q 和 K 的部分维度做旋转。位置不同，旋转角度不同。

RoPE 的直觉：按位置旋转 Q/K，让点积携带相对位置信息



最关键的一点

RoPE 不是简单把一个 position vector 加到 token embedding 上；它把位置信息直接写进 Q/K 的几何关系，从而影响 Attention 分数。

10. 为什么“旋转”能够表达相对位置？

先看二维向量。把向量旋转角度 θ ，本质是乘一个旋转矩阵：

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

若位置 m 的 Q 旋转 θ_m ，位置 n 的 K 旋转 θ_n ，那么它们做点积时，结果会与两者的角度差 $\theta_n - \theta_m$ 有关。角度差又由位置差 $n-m$ 决定。

这就是 RoPE 的漂亮之处

它以一种几何方式，让 Attention 的 $Q \cdot K$ 自然感受到“两个 token 相距多少”。

真实模型会把高维向量分成很多二维对子，并给不同对子使用不同旋转频率。你现在只需掌握这个几何直觉。

11. 一个极简数值例子：位置改变， $Q \cdot K$ 也改变

假设两个二维内容向量本来不变。我们只改变它们所在位置，对 Q/K 应用不同角度旋转。

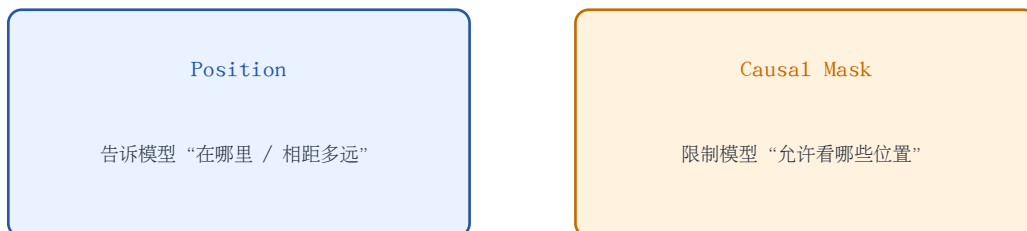
情况	位置差	旋转后点积（直觉）
靠得近	$n-m$ 较小	按内容 + 相对位置共同变化
离得远	$n-m$ 较大	角度差不同，分数也会变化

注意

不要把它简化成“距离越远，分数一定越小”。RoPE
不是固定衰减函数；不同频率维度共同作用，具体关系更复杂。

12. Position Encoding 与 Causal Mask 不是一回事

Position Encoding 与 Causal Mask 解决两个不同问题



一个负责“坐标系”，一个负责“访问权限”。GPT 通常两者都需要。

例如 GPT 预测第 6 个位置时：

- Position Information: 告诉模型各 token 的位置/相对关系；
- Causal Mask: 禁止第 6 个位置看到第 7、8、9……这些“未来 token”。

常见误区

“既然有位置编码，就知道不能看未来”——错误。知道未来 token 在哪里，并不等于被禁止读取它。访问限制需要 Mask。

13. Position Encoding 与 Token Embedding 到底如何相遇？

不同架构处理方式不同。经典绝对位置方法常在网络入口把 token embedding 与 position embedding 相加；RoPE 则主要在 Attention 内部对 Q/K 施加位置相关旋转。

方法	位置信息注入位置	关键对象
Learned absolute	通常输入层	hidden/input representation
Sinusoidal absolute	通常输入层	hidden/input representation
Relative bias	Attention score 阶段	$\text{score}(i,j)$
RoPE	Attention 内部	Q 和 K

所以

“位置编码”不是一个固定模块名称，而是一类“让模型获得序列位置信息”的方法族。

14. 为什么长上下文会让位置问题变得更难？

模型训练时通常只见过某个长度范围。例如只在 4K 或 32K 上训练，后来希望直接扩到更长上下文，就不只是显存问题，还会遇到“位置表示能否外推”的问题。

- 训练长度之外的位置是否仍具有合理几何结构？
- 很远的 token 是否还能稳定建立相对关系？
- 改变 RoPE 频率/缩放方式后，会不会损害短上下文能力？
- 上下文变长后标准 Attention 的 $O(n^2)$ 成本仍然存在。

科研视角

“支持 1M context” 不等于 “1M 内任意位置的信息都能同样可靠地被利用”。位置外推、检索能力、Lost-in-the-Middle、注意力成本是不同问题。

15. RoPE

Scaling：为什么会看到“扩上下文”的各种技巧？

如果模型原来在某个位置范围内学会了 RoPE 的角度模式，直接把位置拉得很长可能进入训练时很少见的角度区域。因此工程上出现许多对位置索引或旋转频率做缩放/插值的办法。

现在只掌握这一层

这些方法的共同目标不是“增加模型记忆”，而是尽量让更长位置范围的 RoPE 表示仍然处于模型可以利用的几何分布中。具体算法以后讲长上下文系统时再深入。

16. ALiBi 是什么？只需要建立最小概念

ALiBi (Attention with Linear Biases) 代表另一条思路：不把位置 embedding 加到输入，而是给 Attention score 加一个随相对距离变化的线性 bias。不同 Head 可以使用不同斜率。

$$\text{score}(i,j) = q_i \cdot k_j - \text{slope}_h \times \text{distance}(i,j)$$

先不用深挖

你现在只需知道：现代位置机制不只有 RoPE；研究者一直在探索“如何让模型更自然地编码距离、顺序并外推到更长上下文”。

17. 位置编码会不会自己理解语法？

不会。位置编码只是提供位置信号。真正把“前面的名词通常是主语”“这个代词可能指向前文实体”等模式学出来的，仍然是 Transformer 参数在训练数据和 Loss 驱动下形成的。

组件	提供什么	不等于什么
Token Embedding	内容起点	完整语义理解
Position Information	位置/距离线索	语法规则本身
Attention	动态信息路由	事实正确性保证
训练参数	从数据中学习模式	人类显式规则库

18. Agent Systems 连接：顺序是一种“接口契约”

当你把 System Prompt、用户消息、工具返回、Memory、多个 Agent 的消息拼进一个上下文时，它们最终都会排成一个 token 序列。序列中的先后、距离、边界，会通过位置机制影响模型内部计算。

Agent Systems 连接

在多 Agent 系统里，“把重要消息放在哪里”“历史记录有多远”“工具结果插在什么位置”并不是纯排版问题，而可能改变模型的注意力路径和可利用性。

- 同样的信息放在上下文开头、中间、末尾，效果可能不同；
- 长历史会把早期关键信息推到更远位置；
- 摘要、Memory Retrieval、Context Engineering 的价值之一，就是重新组织“哪些信息出现在什么位置”。

19. 常见误区集中纠正

误区	正确理解
“Attention 本身知道先后顺序”	没有独立位置信号时，纯 Self-Attention 不具备固定绝对顺序坐标
“Position Encoding 就是给每个词编号”	编号只是概念；实际要转换成向量、旋转或 bias
“RoPE 是把一个向量加到 Embedding”	RoPE 主要对 Q/K 做位置相关旋转
“位置越远 Attention 一定越小”	不成立；具体由内容、位置机制与参数共同决定
“有位置编码就不需要 Causal Mask”	错误；两者解决不同问题
“上下文窗口变长就等于记忆更强”	错误；长度、可检索性、位置外推和注意力成本是不同维度

20. 小实验：亲眼看位置如何改变 Self-Attention

材料包中的 Python 代码使用 NumPy

做三个极小实验：A) 无位置时的置换等变性；B) 加入极简绝对位置后，简单置换对称性被打破；C) 二维 RoPE 旋转如何改变 Q/K 点积。

运行文件

04_position_encoding_demo.py

建议你修改 token 顺序、位置向量、position_m、position_n 和 theta_base，观察输出如何变化。

21. 练习题

- 1) 为什么“狗咬人”和“人咬狗”能说明位置编码的必要性？
- 2) 没有位置编码时，Self-Attention 的置换等变性是什么意思？
- 3) Token Embedding 与 Position Information 分别提供什么？
- 4) Learned Position Embedding 与 Sinusoidal Position Encoding 的核心区别是什么？
- 5) 为什么 Sinusoidal 要使用多种频率？
- 6) 绝对位置与相对位置分别回答什么问题？
- 7) RoPE 主要作用在什么对象上？为什么旋转能带来相对位置信息？
- 8) Position Encoding 与 Causal Mask 的区别是什么？
- 9) 为什么“上下文长度增加”不等于“模型一定能可靠利用更远的信息”？
- 10) 在 Agent Systems 中，为什么消息在上下文中的位置可能影响系统表现？

22. 练习答案

- 1) 词集合相同但顺序改变就导致语义角色变化，说明模型必须有顺序/位置线索。
- 2) 把输入 token 整体重新排列，输出会按相同方式重新排列；架构没有固定绝对位置坐标。
- 3) Token Embedding 提供内容表示起点；Position Information 提供位置、顺序或相对距离线索。
- 4) Learned PE 是可训练参数；Sinusoidal PE 按确定的 $\sin/\cos$ 公式生成，不需要学习。
- 5) 不同频率提供多尺度位置特征，组合后形成更丰富的位置指纹。
- 6) 绝对位置回答“我在第几位”；相对位置回答“你离我多远、在我前还是后”。
- 7) RoPE 主要旋转 Q/K。两个位置对应不同角度，其点积会与角度差、进而与相对位置有关。
- 8) Position 提供坐标/距离；Causal Mask 规定哪些位置允许被访问。
- 9) 还存在位置外推、检索能力、注意力成本、训练分布与 Lost-in-the-Middle 等问题。
- 10) 消息最终成为 token 序列，不同位置和距离会改变模型内部可用的位置信号与注意力关系。

23. 本讲总结：把整条链串起来

Token $\rightarrow$ Token Embedding + Position Information (或在 Attention 内注入位置) $\rightarrow$ Q/K/V $\rightarrow$ Attention Score
同时受内容与位置影响 $\rightarrow$ 上下文文化表示

最低闭环能力

看到论文里的 Positional Encoding、Relative Position、RoPE、ALiBi
时，至少能问：位置被注入在哪里？表达绝对位置还是相对距离？会怎样影响 Q/K/Attention score？训练长度之外如何处理？

下一讲

第11讲将把 Embedding、Multi-Head Attention、Residual、LayerNorm、FFN 全部拼起来：一个完整 Transformer Block 到底怎样流动？

参考原始工作：Vaswani et al., Attention Is All You Need (2017); Su et al., RoFormer / Rotary Position Embedding (2021)。